

Compte Rendu Mini-Projet FPGA

Noah ALVES

Martin GARNAULT

Contexte

L'objectif de ce projet est d'implémenter sur une carte Basys3, construite autour d'un FPGA Artix7, une centrale de pilotage pour trains miniatures. Ces trains réagissent au protocole DCC, qui permet de :

- contrôler individuellement chaque train grâce à un système d'adresse.
- contrôler la vitesse des trains.
- contrôler la direction des trains.
- activer des fonctions sonores et visuelles comme des annonces, ou allumer les phares.

L'implémentation finale contiendra toutes ces fonctionnalités au travers d'une interface homme-machine simple d'utilisation, exploitant l'afficheur 7 segments et les 5 boutons poussoirs de la carte Basys3.

Les commandes sont transmises sous forme de trames numériques à travers les rails du train. Le détail du fonctionnement sera détaillé au fur et à mesure de ce document.

Architecture globale

La structure de la base de contrôle peut être divisée en deux. Une partie matérielle, et une partie logicielle.

La partie matérielle se compose d'une IP (*Intellectual Property*) nommée *CENTRALE_DCC*, qui implémente les spécifications techniques du protocole DCC, et d'un microprocesseur softcore Microblaze, qui est chargé d'orchestrer le système et de gérer la partie interface homme-machine.

La partie logicielle, écrite en langage C pour le MicroBlaze, implémente la gestion des différents menus de fonctions, l'affichage du texte, mais aussi la génération au bit près de la commande demandée par l'utilisateur.

Partie Hardware

Dans un premier temps, nous avons travaillé à construire une IP (*Intellectual Property*) nommée *CENTRALE_DCC*.

Cette IP facilite l'implémentation du protocole DCC du point de vue du microblaze, en réalisant l'abstraction de toute la couche matérielle (génération de bits avec timings précis, registres à décalage, etc...), exposant simplement des registres où la commande pourra être placée, puis automatiquement envoyée.

Voici la structure globale de l'IP *Centrale DCC* :

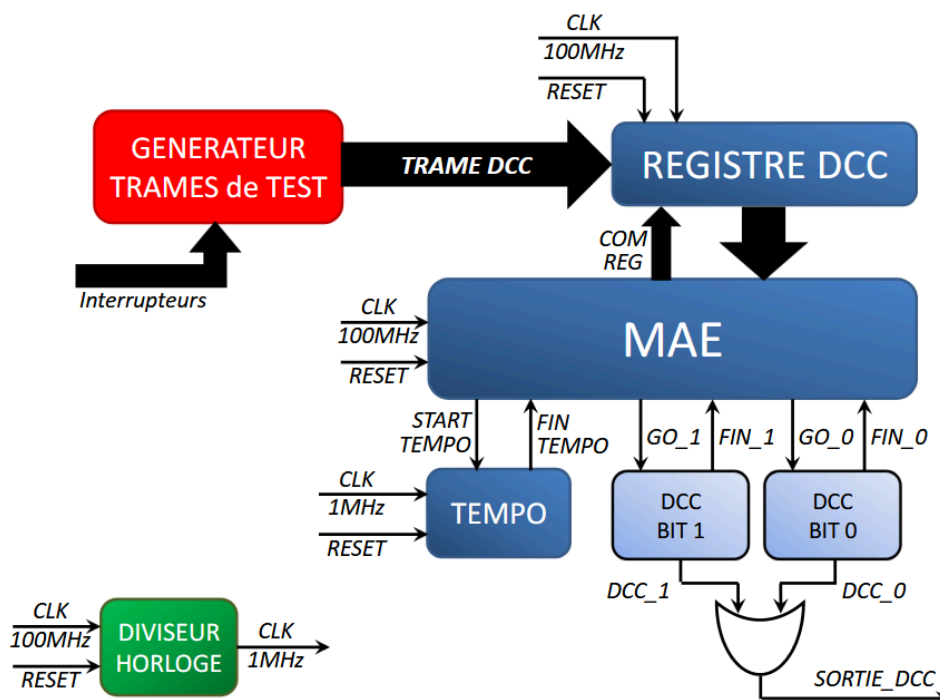


Figure 3 – Architecture de la Centrale DCC

L'architecture repose sur une Machine à États synchrone centrale et un certain nombre d'éléments périphériques. Nous allons dans ce document détailler l'implémentation de chacun des éléments du système.

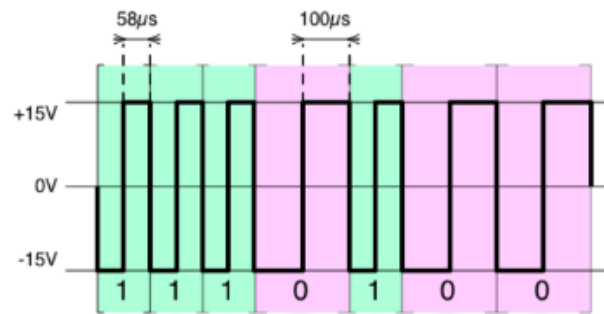
Génération de bits : DCC BIT 1|0

Nous avons commencé l'implémentation avec le générateur de symbole pour le protocole DCC.

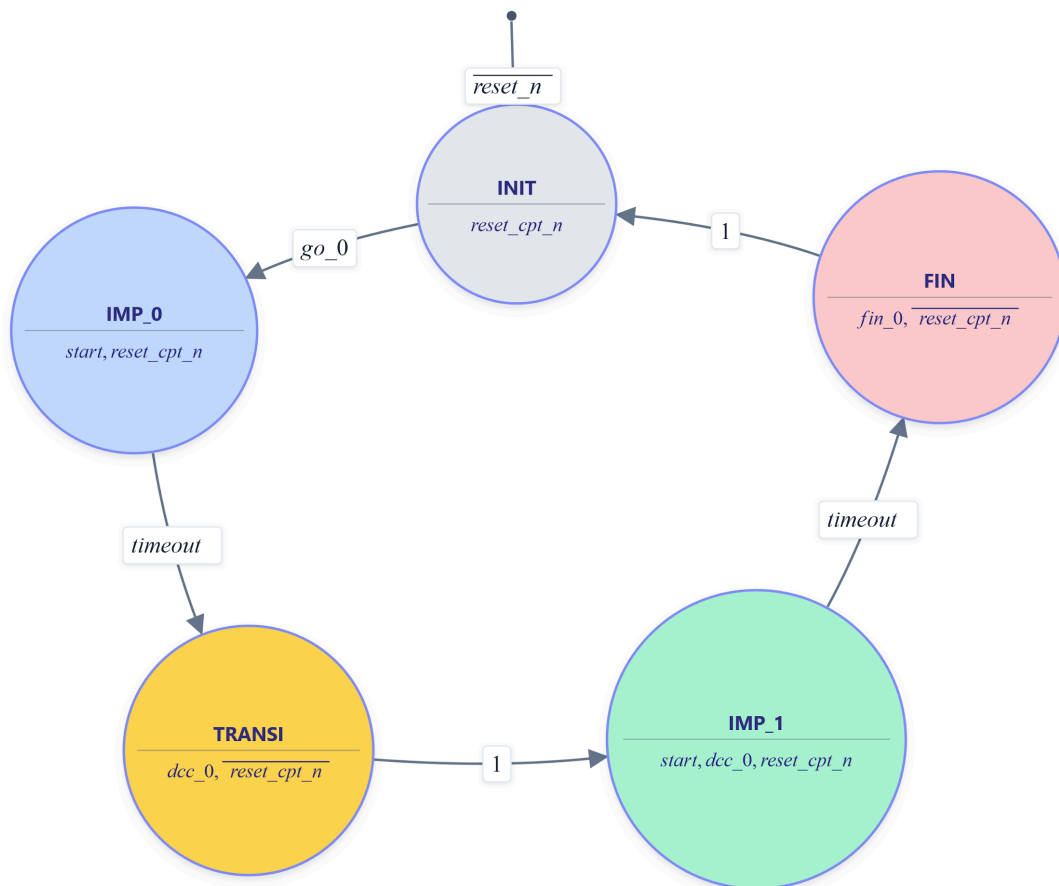
Ces deux modules (un pour la génération de bit 1, un pour la génération de bit 0), prennent simplement en entrée un signal *GO_X* qui lance le module et génère le symbole correspondant. Lorsque la génération du bit est terminée, le module active le signal *FIN_X*.

Voici la forme d'onde et les timings à respecter pour envoyer un bit 1 ou un bit 0.

Bit	Représentation
0	Impulsion à 0 de 100 μ s puis Impulsion à 1 de 100 μ s
1	Impulsion à 0 de 58 μ s puis Impulsion à 1 de 58 μ s



Les générateurs de bit reposent en interne sur une machine à état et sur un compteur. Voici le graphe d'état de la machine à état pour le bit 0 :



Le compteur est cadencé par une horloge à 1 MHz tandis que la MAE est cadencée à 100MHz.

Le signal **start** est un signal qui lance le compteur, qui s'incrémente chaque microseconde. Quand le compteur atteint sa valeur finale (comme 100 μ s pour le bit 0), il active le signal **timeout**.

L'état de transition permet de remettre à zero le compteur.

Comme la période à l'état haut et la période à l'état bas sont symétriques pour un bit donné, on peut lancer deux fois le même compteur.

Ainsi, ce module permet de générer un bit 0 ou 1 à la demande.

Testbench

Pour valider cette implémentation un testbench avec contrôle automatisé de la durée de chaque période a été construit.

Pour deux bits '1' consécutifs, voici le log (nettoyé des logs de Vivado pour la lecture) :

```
Note: === DEBUT DU TEST : Generation de deux bits '1' consecutifs ===
Time: 10 us Iteration: 0
Note: SUCCES IMP_0 : Duree = 57490 ns (Delta: 510 ns)
Time: 67505 ns Iteration: 3
Note: SUCCES IMP_1 : Duree = 58000 ns (Delta: 0 ns)
Time: 125505 ns Iteration: 3 Note: SUCCES IMP_0 : Duree = 57980 ns (Delta: 20 ns)
Time: 183505 ns Iteration: 3 Note: SUCCES IMP_1 : Duree = 58000 ns (Delta: 0 ns)
Time: 241505 ns Iteration: 3 Note: === FIN DE SIMULATION ===
```

Le premier delta de 510ns est dû à la première synchronisation des horloges 1M et 100M. Autrement, le delta est de 20ns pour l'impulsion '0' (c'est à dire la partie à 0V du symbole) et de 0ns pour l'impulsion '1' (c'est à dire la partie à VDD du symbole).

Il serait possible de rendre ces timings complètement exacts en modifiant la MAE mais par soucis d'efficacité nous avons continué à implémenter d'autres modules. Ces deltas de quelques dizaines de nanosecondes rentrent tout à fait dans la tolérance de marge du protocole DCC. Des résultats similaires ont été constatés pour le bit '0'.

Registre DCC

Ensuite, nous avons poursuivi la construction de l'IP Centrale DCC par le registre DCC. Ce module fait l'interface entre le bus AXI et la machine à états (MAE) principale. Il réceptionne la trame DCC globale transmise par le bus, puis en extrait et fournit les bits un par un à la MAE principale via un mécanisme de registre à décalage. Le bit de poids fort (MSB) du registre est ainsi exposé en continu pour dicter l'état de la MAE.

Ainsi, dans notre implémentation, le registre DCC réagit à deux commandes :

- shift : le contenu du registre se décale d'un bit vers la gauche.
- load : le registre se charge avec le contenu de son entrée Trame_DCC.

Le registre expose deux sorties à la MAE :

- msb : le bit du poids fort du registre
- end_shift : signal qui s'active lorsque le registre est complètement décalé.

La taille du registre est un élément demandant une attention particulière. En effet, une des particularité du protocole DCC est que certaines fonctions se codent sur 1 octet, et d'autres sur 2 octets. Ainsi, la taille de la commande peut varier. Il nous faudrait donc un registre de taille dynamique ! Cependant, chaque trame DCC débute par un préambule dont la taille est de minimum 14 bits. Le protocole admet sans problème un préambule de taille supérieure.

Composition de la Trame	Détail
Préambule	Suite d'au moins 14 bits à 1
Start Bit	1 bit à 0
Champ d'adresse	Adresse de la locomotive à commander (1 octet)
Start Bit	1 bit à 0
Champ de commande	Commande envoyée au train (de 1 à 2 octets*)
Start Bit	1 bit à 0
Champ de contrôle	XOR entre les octets des deux champs précédents, (1 octet). Permet de détecter d'éventuelles erreurs de transmission
Stop Bit	1 bit à 1

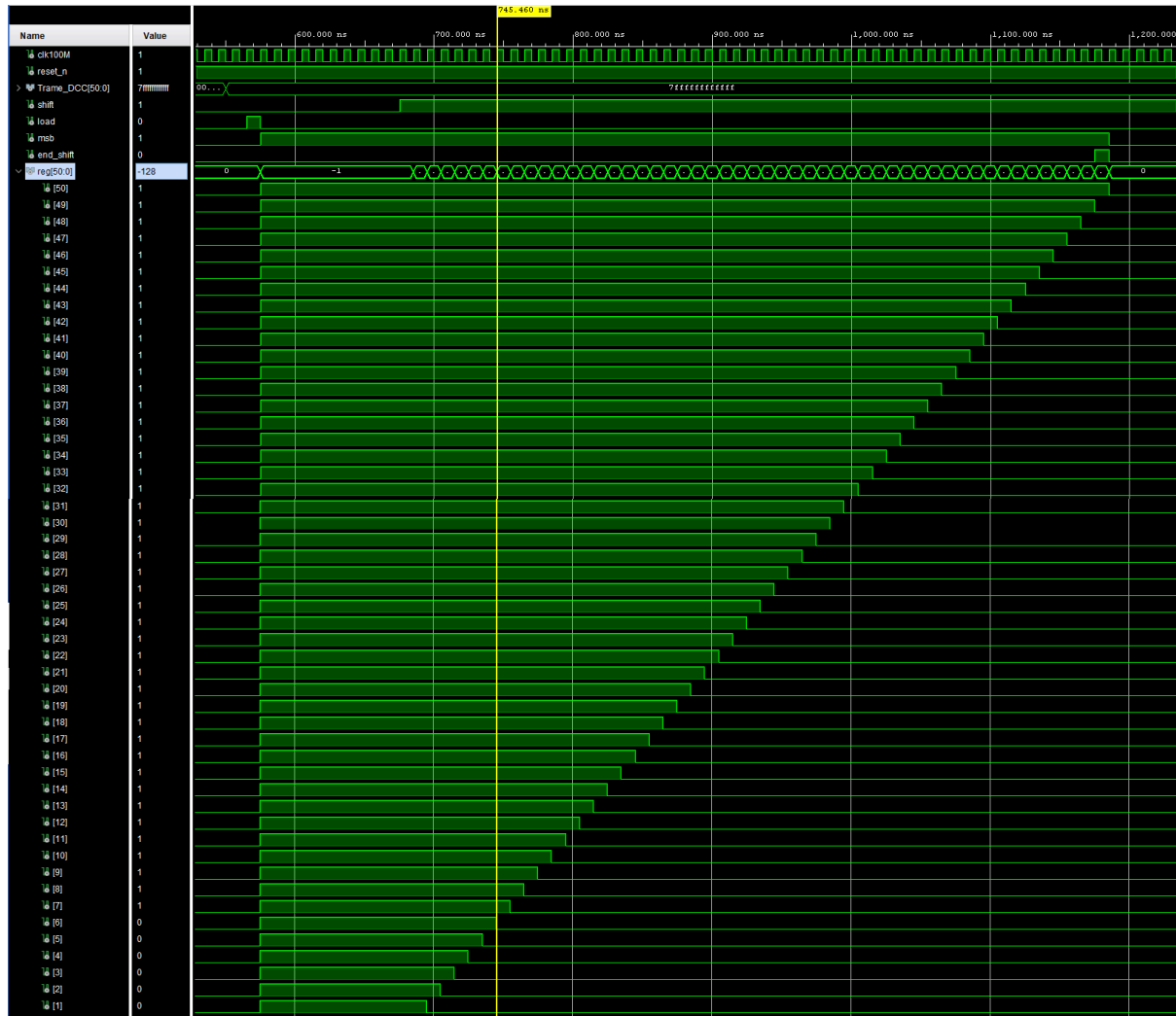
Dans le but de simplifier l'implémentation du registre DCC, nous avons décidé d'augmenter la taille du préambule pour les commandes sur 1 octet afin d'avoir une taille fixe de trame.

La taille du registre DCC est donc :

$$(14)+1+(8)+1+((8)+1+(8))+1+(8)+1 = 51 \text{ bits}$$

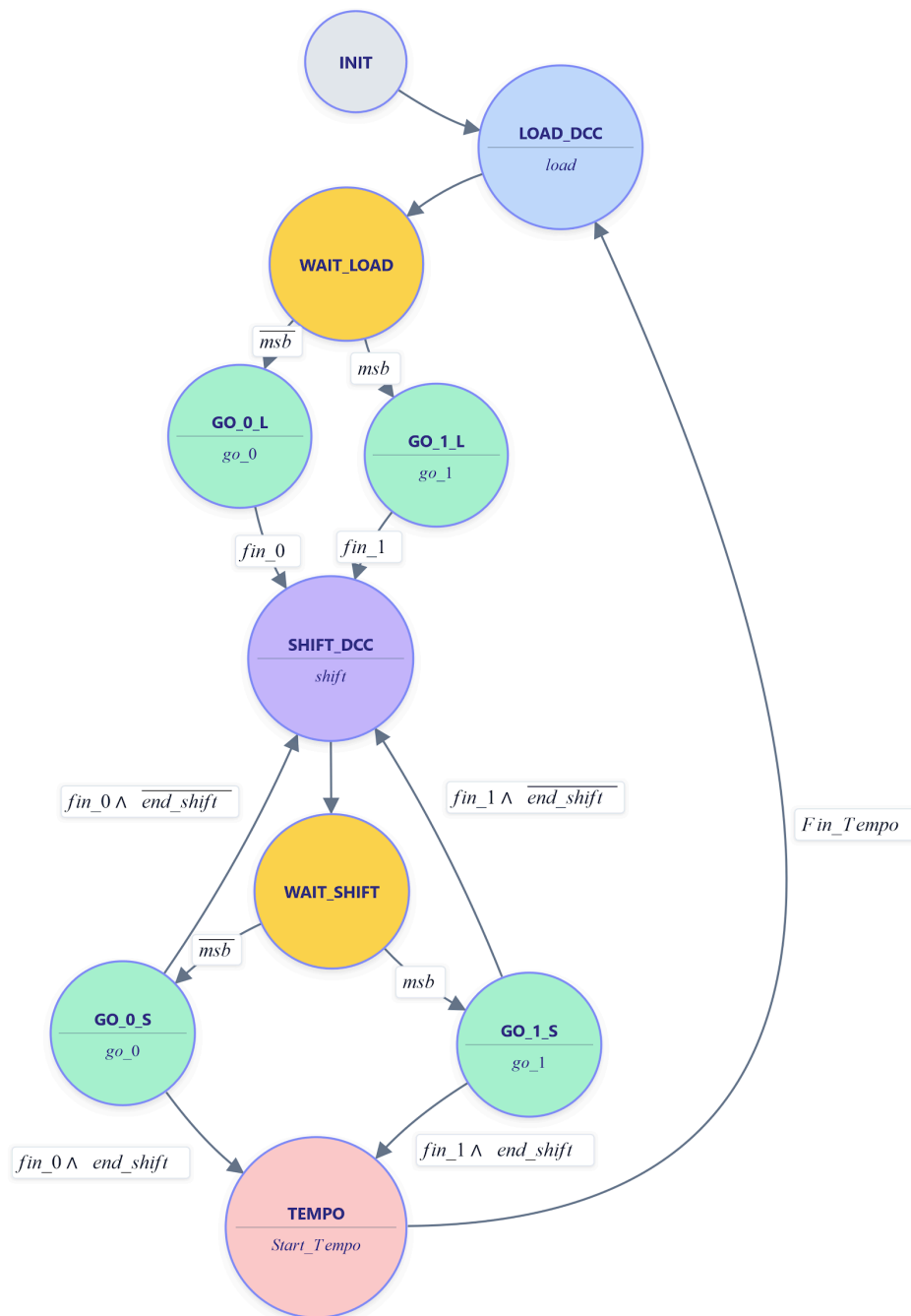
Testbench

Pour tester notre registre nous l'avons simplement rempli de '1' et avons mis le signal **shift** à '1'. Il est ainsi simple de voir le registre se vider, du bit [0] au bit [50]. Sur l'image, le bit [0] est coupé, désolé...



Machine à état principale

L'IP DCC est orchestrée par une machine à état principale dont voici le graphe d'état.



Le cœur du fonctionnement est une boucle autour de l'état **SHIFT_DCC**. Tant que le registre n'est pas complètement vidé, on envoie *shift* et on demande aux générateurs de symboles d'envoyer le symbole correspondant.

Une fois que tous les bits ont été transmis, le protocole DCC impose une pause de 6ms. Pour

intégrer ce délai dans la MAE, on utilise le module *TEMPO*, qui nous a été fourni, et qui active le signal *Fin_Tempo* une fois que les 6ms sont écoulées. A ce moment là, la machine à état peut redémarrer et demander au registre de charger la trame DCC.

Des états d'attente ont été insérés après chaque commande envoyée au registre pour laisser le temps à ce dernier d'effectuer l'action. Autrement, le bit présenté au cycle juste après la commande n'est pas le bon.

Cette machine à état permet donc d'envoyer en boucle, selon les spécifications du protocole DCC, la trame présentée au registre DCC.

Intégration dans TOP_DCC

Tous ces modules doivent ensuite être assemblés dans un module plus haut niveau : TOP_DCC.

Dans ce module, on instancie chaque module, et on les relie entre eux selon le schéma présenté plus haut. On ne doit pas oublier la porte OU en sortie des deux générateurs de bits.

Pour tester l'IP, nous avons complété le module GENERATEUR TRAME DE TEST, dont le rôle est d'exposer des trames valides au registre DCC. Dans l'IP finale, ce module sera remplacée par le bus AXI.

Après avoir renseigné un fichier de contrainte matérielle, nous avons pu directement mesurer la sortie *SORTIE_DCC* à l'oscilloscope.

Testbench

Pour tester notre système, nous avons écrits un testbench qui simule les interrupteurs en entrée du module générateur de trames. Ce dernier envoie des trames valides pour certaines fonctions comme la marche avant, l'allumage des phares, etc...

De plus, nous avons mis en place un simple décodeur de trame qui mesure la largeur de l'impulsion 0->1->0 pour déduire le bit envoyé.

```
-- mesure de la longueur de l'impulsion -> déduction du bit
wait until Sortie_DCC = '1' or sim_done;
t_rise := now;
wait until Sortie_DCC = '0' or sim_done;
pulse_width := now - t_rise;
-- déduction du bit
if pulse_width < 75 us then
    frame_reg(bit_count) := '1';
else
    frame_reg(bit_count) := '0';
end if;
```

Ainsi, nous pouvons décoder la trame que notre IP génère en direct. En la comparant avec la trame théorique, nous pouvons valider notre implémentation matérielle.

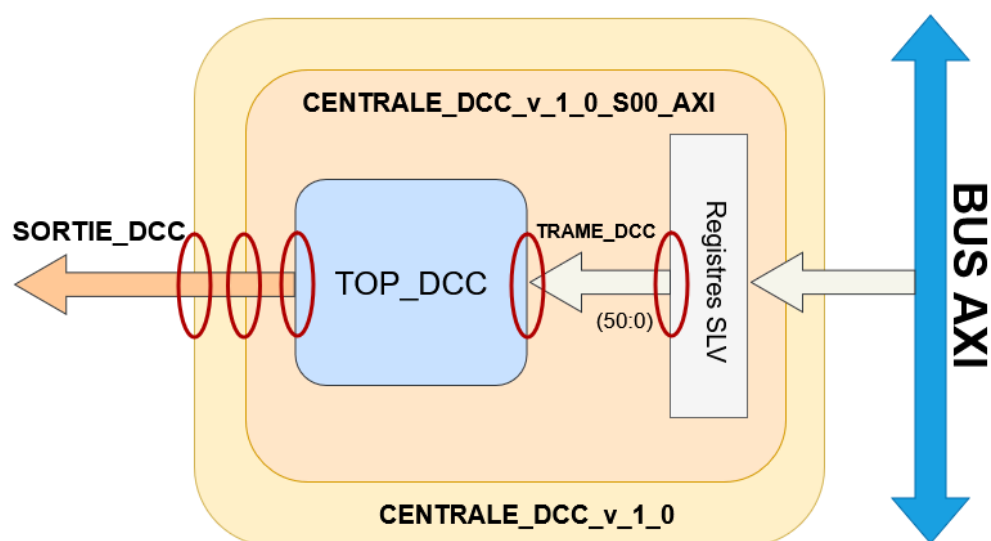
Voici les logs que nous obtenons à la fin de notre simulation :

```
Note: === TEST 1 : Interrupteurs a 0 (Arret) ===  
Time: 20 us  
Note: TRAME OK  
Time: 7689505 ns  
Note: === TEST 2 : Interrupteur 7 a 1 (Marche avant) ===  
Time: 9689505 ns  
Note: TRAME OK  
Time: 21200505 ns  
Note: === TEST 3 : Interrupteur 5 a 1 (Phares) ===  
Time: 23200505 ns  
Note: TRAME OK  
Time: 34879505 ns  
Note: === FIN DES TESTS SYSTEME : TOUTES LES TRAMES SONT VALIDEES ===
```

Lien avec le bus AXI

À cette étape, notre système n'est pas encore prêt pour être utilisé comme une IP classique. Il faut relier la fonctionnalité de l'IP au bus AXI.

Voici un diagramme illustrant la structure créée par Vivado à la création d'une nouvelle IP compatible avec le bus AXI4.



Deux fichiers VHDL sont créés :

- Centrale_DCC_v_1_0_SOO_AXI, qui instancie notre module TOP_DCC, et crée des registres SLV dans lesquels nous écriront pour utiliser notre IP.
- Centrale_DCC_v_1_0, qui instancie Centrale_DCC_v_1_0_SOO_AXI en l'interfacant avec le bus AXI.

Il nous reste tout de même quelques connexions internes à réaliser à la main. Elles sont entourées en rouge dans le diagramme.

Tout d'abord nous avons dû relier les registres SLV créés avec TOP_DCC. Pour cela, nous avons ajouté les lignes suivantes dans l'architecture du fichier Centrale_DCC_v_1_0_SOO_AXI.vhd

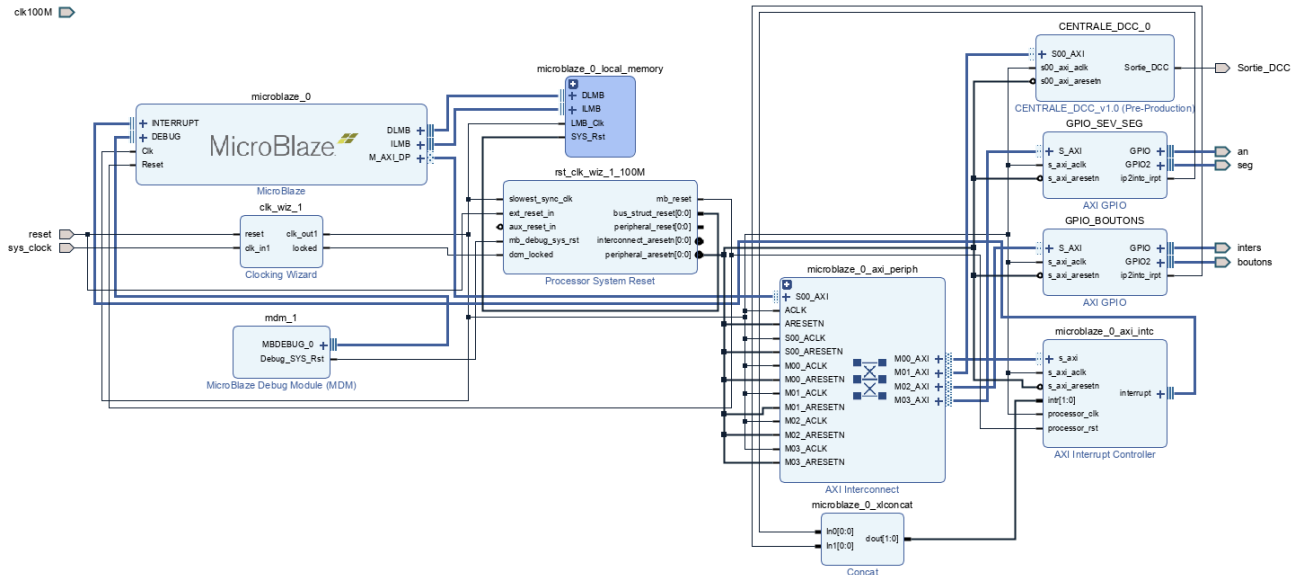
```
-- écrire l'assignation entre les registres slv et la trame.
Trame_DCC(50 downto 19) <= slv_reg0;
Trame_DCC(18 downto 0) <= slv_reg1(18 downto 0);
```

Pour transmettre la trame à l'IP, il faudra donc écrire les 32 premiers bits de la trame dans le reg0 et le reste dans le reg1.

Ensuite, il a fallu créer le port de sortie SORTIE_DCC dans les fichiers Centrale_DCC_v_1_0_SOO_AXI.vhd et Centrale_DCC_v_1_0.vhd et réaliser le port map avec Top_DCC.

Block-design

Une fois notre IP Centrale DCC complète et fonctionnelle, nous avons construit le block-design global de la plateforme matérielle.



Comme présenté dans la section *Architecture Globale*, nous avons associé cette IP à un processeur soft-core Microblaze. Afin de pouvoir interagir avec les boutons poussoirs et les l'afficheur 7 segments, nous avons rajouté des blocks GPIO à l'architecture.

On lit sur ce diagramme :

- Le processeur **MicroBlaze**
- Un block **Clocking_Wizard** généré par Vivado et qui gère automatiquement l'horloge
- Un certain nombre de périphériques AXI Microblaze
- 2 blocks GPIO : **GPIO_SEV_SEG** et **GPIO_BOUTONS**
- Notre IP **CENTRALE_DCC**

Ce block design constitue l'architecture matérielle de notre station de contrôle de trains DCC.

Après avoir exporté cette configuration hardware pour l'environnement Vitis, nous sommes prêts à écrire du code C pour le MicroBlaze.

Note sur les simulations post-synthèse

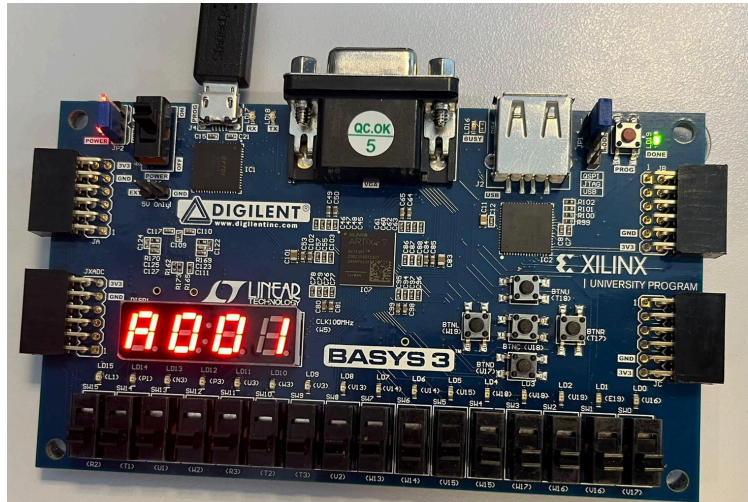
Nous n'avons pas réalisé de simulations post-synthèse pour des raisons de temps. Pour être certain que notre design reste synthétisable, à chaque fin de journée, nous avons lancé une synthèse afin de vérifier qu'aucune erreur ne s'était glissée dans la description matérielle.

Partie Software

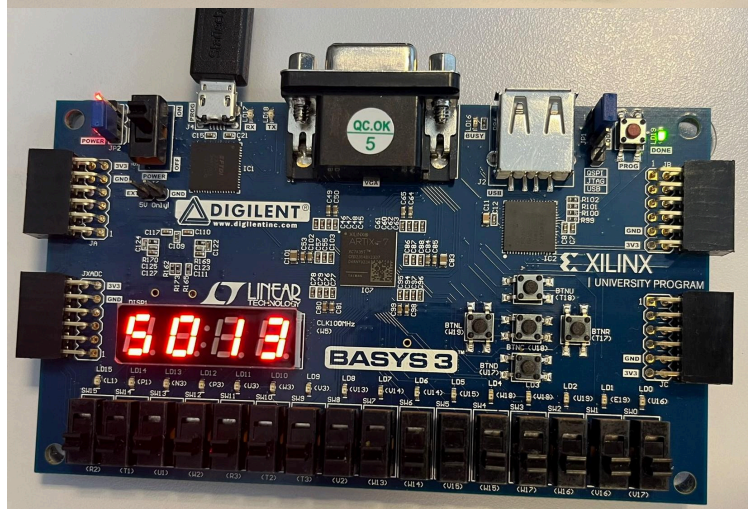
Le cœur de notre architecture logicielle, situé dans le fichier main.c, repose sur une boucle infinie while(1) exécutée de façon séquentielle. Elle s'occupe de mettre à jour en continu l'affichage 7-segments et vérifie l'état des boutons toutes les 10ms. Ce mécanisme permet de réagir aux actions de l'utilisateur pour changer de menu, modifier une valeur ou envoyer une trame DCC.

Nous avons conçu l'interface de contrôle autour de quatre menus :

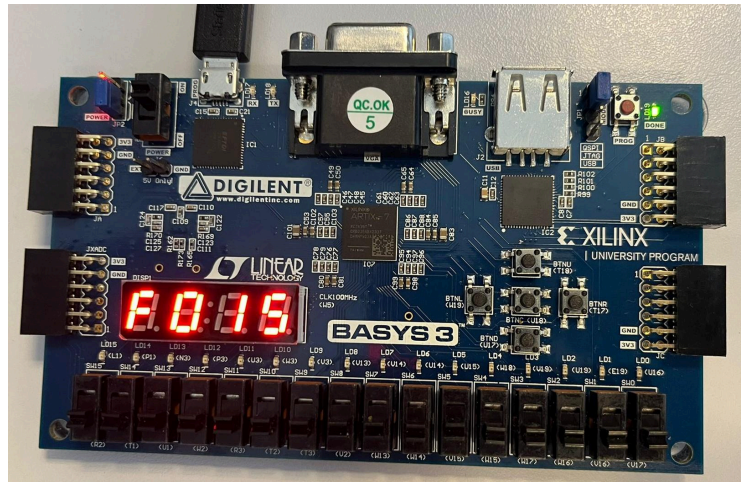
Le menu 0 (Adresse) : Il est dédié au choix de l'adresse du train, sur une plage de 0 à 8. Les boutons haut et bas servent respectivement à incrémenter et décrémenter cette valeur. Une fois l'adresse choisie, l'utilisateur valide sa sélection par l'appui sur le bouton central. Le passage au menu suivant s'effectue ensuite via le bouton droit.



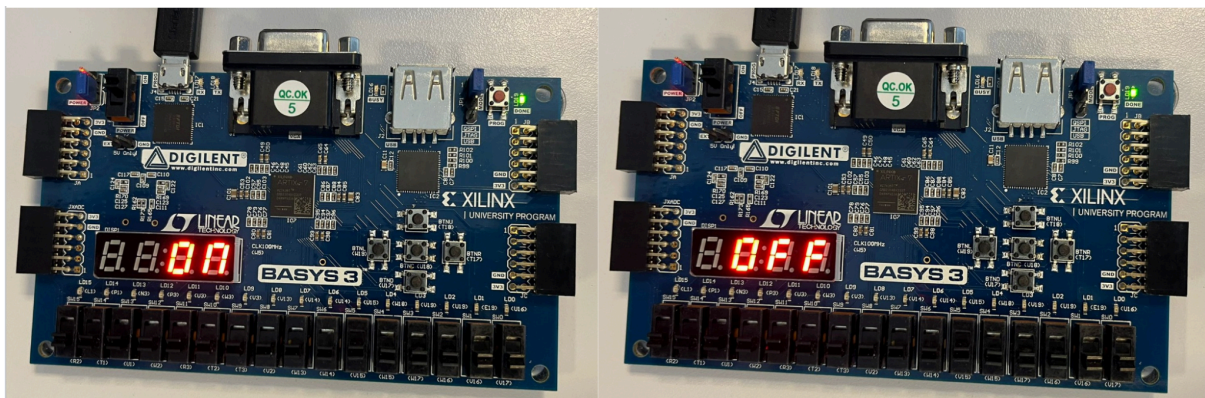
Le menu 1 (Vitesse) : Il contrôle l'allure du train avec des valeurs allant de -28 à 28, les nombres négatifs activent la marche arrière. L'envoi de la trame de commande vers les registres n'est déclenché que lors de l'appui du bouton central.



Le menu 2 (Fonction) : Il gère les fonctionnalités du train, numérotées de 0 à 20. Si une fonction de type one-shot est choisie, la trame est transmise puis automatiquement interrompue quelques millisecondes plus tard.



Le menu 3 (ON/OFF) : Dans le cas d'une fonction continue (non one-shot), l'affichage bascule directement dans ce menu. L'utilisateur y détermine l'état de la fonction (allumée ou éteinte) à l'aide des boutons verticaux. L'envoi de cette commande redirige automatiquement l'utilisateur vers le menu 2.



Dans la suite de cette section, nous détaillerons les fonctions dédiées à :

- L'actualisation de l'affichage 7 segments.
- La configuration d'une trame.
- L'envoi d'une trame.

Actualisation de l'affichage 7 segments

L'affichage 7-segments de notre carte dispose de quatre digits. Le but de la fonction `write_sev_seg` est d'allumer qu'un seul digit à la fois, en les activant successivement à une fréquence suffisamment élevée pour que l'utilisateur perçoive un affichage continu. L'argument central est le pointeur `char* s`, qui permet de transmettre par référence la chaîne de caractères à afficher (par exemple "A001" pour l'adresse ou "S-13" pour la vitesse). Cette chaîne, correspondant à la variable globale `string_to_display` dans notre programme, qui est actualisée dynamiquement dès que l'utilisateur change de menu ou modifie une valeur.

Configuration d'une trame

Dès que l'utilisateur se situe dans les menus 2,3 et 4 et qu'il appuie sur le bouton central, le programme déclenche l'appel d'une fonction de configuration. Il s'agit de `calculate_trame_speed()` si l'on se trouve dans le menu 2, ou de `calculate_trame_func()` pour les menus 3 et 4. Ces deux fonctions prennent comme paramètre un pointeur sur la variable locale *trame* du main.

La fonction `calculate_trame_speed()`, permet de configurer la trame de vitesse d'une locomotive. En utilisant la sélection de l'utilisateur (stockée dans la variable globale *speed*) comme index, le programme extrait le mot binaire de 5 bits approprié depuis la table de correspondance du fichier `speed_table.h`.

La commande de vitesse est encodée sur un octet structuré selon ce format binaire 01DXXXXX. Le bit D détermine la direction du train : 0 déclenche la marche arrière (vitesse négative), tandis qu'un 1 ordonne la marche avant. Les cinq bits de poids faible (XXXXX) reçoivent la valeur binaire extraite du tableau.

La fonction `calculate_trame_func()`, permet de configurer une trame d'activation (ON) ou de désactivation (OFF) d'une fonction d'une locomotive. Elle contient au préalable dans la variable globale *func*, le numéro de la fonctionnalité sélectionnée par l'utilisateur. Selon ce choix et l'état souhaité, le programme extrait le motif binaire approprié depuis les tables de correspondance du fichier `func_table.h`, afin de remplir le champ de commande de la trame DCC.

A la fin de leur exécution, ces deux fonctions réalisent l'assemblage de la trame DCC, encodée sur 51 bits. Cette construction est réalisée grâce à une succession de masques et de décalages binaires. Ces opérations permettent de concaténer le préambule, l'adresse de la locomotive (définie dans la variable globale *addr* du menu 0), le champ de commande spécifique, et enfin l'octet de contrôle de la trame. Le résultat final est alors stocké dans la variable locale *trame* du main, afin de procéder par la suite à son envoi.

Envoi d'une trame

La fonction `send_trame()` assure l'envoi d'une trame vers la locomotive cible. Exécutée juste après l'une des fonctions de configuration précédemment étudiées, elle prend en paramètre un pointeur sur la variable locale *trame*. La première étape de cette fonction consiste à découper notre trame complète de 51 bits en deux parties. En effet, comme expliqué dans la section Hardware, la transmission des données vers l'IP CENTRALE_DCC nécessite d'écrire les 32 premiers bits de notre trame dans le registre 0, et les bits restants dans le registre 1.

Après avoir découpé la trame en deux sous-ensembles, le programme effectue alors deux écritures en mémoire aux adresses physiques allouées à ces deux registres. Cette opération est réalisée grâce à la macro `CENTRALE_DCC_mWriteReg`, qui transfère les données à l'IP matérielle qui se chargera ensuite, de son côté, de l'envoi physique sur les voies.

Conclusion

Nous avons avec succès implémenté une plateforme de contrôle de trains miniatures sur FPGA. La construction de cette plateforme a été divisée en deux grandes parties :

- le développement d'une IP **CENTRALE_DCC**, qui s'occupe de la génération de symboles et des timings matériels associés.
- le développement de code C sur un microprocesseur soft-core **Microblaze**, qui se charge de la génération du contenu de la trame demandée et de l'interface utilisateur.

Il aurait été intéressant de développer l'IP de contrôle pour l'accéléromètre, mais nous avons manqué de temps pour ajouter cette fonctionnalité.